



پاسخ تمرین اول

پاسخ سوال ۱: تحلیل انتشار رو به جلو، عقب و مرگ نورون‌ها

الف) محاسبه انتشار رو به جلو (Forward Pass): ابتدا پیش‌فعال‌سازی لایه اول ($z^{[1]}$) را محاسبه می‌کنیم:

$$z^{[1]} = W^{[1]}x + b^{[1]} = \begin{bmatrix} 0.5 & 1.0 \\ 0.5 & -2.0 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} + \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} (0.5)(2) + (1.0)(-1) \\ (0.5)(2) + (-2.0)(-1) \end{bmatrix} + \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix}$$

$$z^{[1]} = \begin{bmatrix} 0 \\ 3 \end{bmatrix} + \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 3.5 \end{bmatrix}$$

سپس تابع فعال‌سازی ReLU را اعمال می‌کنیم ($a = \max(0, z)$):

$$a^{[1]} = \begin{bmatrix} \max(0, -1.5) \\ \max(0, 3.5) \end{bmatrix} = \begin{bmatrix} 0 \\ 3.5 \end{bmatrix}$$

حال پیش‌فعال‌سازی لایه خروجی ($z^{[2]}$) را محاسبه می‌کنیم:

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} = \begin{bmatrix} 1.0 & -1.0 \end{bmatrix} \begin{bmatrix} 0 \\ 3.5 \end{bmatrix} + 0 = -3.5$$

در نهایت، خروجی پیش‌بینی شده ($\hat{y}$) با تابع Sigmoid به دست می‌آید:

$$\hat{y} = \sigma(-3.5) = \frac{1}{1 + e^{3.5}} \approx \frac{1}{1 + 33.11} \approx 0.029$$

ب) گرادینان و چالش تله‌دار: تابع هزینه کل برابر است با: $L_{total} = L_{BCE} + \lambda \sum w_{ij}^2$. برای محاسبه گرادینان نسبت به $W_{1,1}^{[1]}$ ، باید اثر هر دو بخش (خطای پیش‌بینی و منظم‌سازی) را در نظر بگیریم. از آنجا که پیش‌فعال‌سازی نورون اول $z_1^{[1]} = -1.5 < 0$ است، مشتق تابع ReLU برای این نورون برابر با صفر خواهد بود. طبق قاعده زنجیره‌ای (Chain Rule)، گرادینان خطای L_{BCE} که به این وزن پس‌انتشار می‌یابد، دقیقاً صفر می‌شود:

$$\frac{\partial L_{BCE}}{\partial W_{1,1}^{[1]}} = 0$$

این پدیده همان مرگ ReLU (Dying ReLU) است. اما تله اینجاست: گرادینان کل صفر نیست! زیرا ترم منظم‌سازی L2 همچنان وجود دارد. مشتق بخش منظم‌سازی نسبت به این وزن برابر $2\lambda W_{1,1}^{[1]}$ است:

$$\frac{\partial L_{total}}{\partial W_{1,1}^{[1]}} = 0 + 2(0.1)(0.5) = 0.1$$

بنابراین، الگوریتم بهینه‌سازی، این وزن را صرفاً به دلیل اثر منظم‌سازی (Weight Decay) به سمت صفر کاهش می‌دهد، اما هیچ اطلاعات مفیدی از خطای پیش‌بینی شبکه دریافت نمی‌کند.

پاسخ سوال ۲: بحران محوشدگی گرادیان و اثرات ساختاری

الف) اثبات محوشدگی گرادیان: طبق قاعده زنجیره‌ای در پس‌انتشار برای L لایه، گرادیان لایه‌های ابتدایی متناسب با ضرب مشتق توابع فعال‌سازی لایه‌های بعدی است: $\frac{\partial L}{\partial W^{[l]}} \propto \prod_{l=1}^L \sigma'(z^{[l]})$. می‌دانیم که ماکزیم مقدار مشتق تابع Sigmoid برابر با 0.25 است ($\sigma'(z) \leq 0.25$). برای 50 لایه، این ضرب حداقل به کوچکی 0.25^{50} خواهد بود:

$$0.25^{50} \approx 7.8 \times 10^{-31}$$

این عدد عملاً در محاسبات کامپیوتری صفر در نظر گرفته می‌شود، در نتیجه گرادیان به لایه‌های اولیه نمی‌رسد و شبکه‌های عمیق در لایه‌های ابتدایی چیزی یاد نمی‌گیرند.

ب) راه‌اندازی با اوزان بزرگ و نقش Batch Normalization: تابع ReLU مشکل محوشدگی را در سمت مثبت حل می‌کند (مشتق برابر ۱ است). اما اگر اوزان بسیار بزرگ باشند (مثل مقادیر ۱۰ تا ۲۰)، مقادیر z در هر لایه به شدت بزرگ می‌شوند. از آنجا که ReLU مقادیر مثبت را محدود نمی‌کند، این مقادیر در طول لایه‌ها به صورت نمایی رشد کرده و منجر به انفجار گرادیان‌ها و خروجی‌های بی‌نهایت (NaN) می‌شوند. Batch Normalization این فاجعه را با نرمال‌سازی مقادیر پیش‌فعال‌سازی در هر مینی‌بچ کنترل می‌کند:

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

این کار باعث می‌شود مستقل از مقیاس اوزان اولیه، توزیع ورودی‌های هر لایه همواره در یک محدوده معقول (میانگین صفر و واریانس یک) باقی بماند و از انفجار یا محوشدگی شدید جلوگیری کند.

پاسخ سوال ۳: چالش کدنویسی (بدون حلقه)

```
1 def backward_and_adam_update(X, A1, W2, dZ2, W1, b1, m, t, m_t, v_t):
2
3     # 1. Compute dZ1 (Gradient of ReLU pre-activation)
4     # W2.T @ dZ2 backpropagates the error. (A1 > 0) is the derivative of
5     # ReLU.
6     dZ1 = np.dot(W2.T, dZ2) * (A1 > 0)
7
8     # 2. Compute dW1 and db1 explicitly
9     dW1 = (1 / m) * np.dot(dZ1, X.T)
10    db1 = (1 / m) * np.sum(dZ1, axis=1, keepdims=True)
11
12    # 3. Adam Optimizer Updates for W1
13    beta1, beta2, lr, epsilon = 0.9, 0.999, 0.01, 1e-8
14
15    # Update biased first and second moment estimates
16    m_t_new = beta1 * m_t + (1 - beta1) * dW1
17    v_t_new = beta2 * v_t + (1 - beta2) * (dW1 ** 2)
18
19    # Compute bias-corrected first and second moment estimates
20    m_t_hat = m_t_new / (1 - beta1 ** t)
```

```

20 v_t_hat = v_t_new / (1 - beta2 ** t)
21
22 # Update weights
23 W1_updated = W1 - lr * (m_t_hat / (np.sqrt(v_t_hat) + epsilon))
24
25 return W1_updated, m_t_new, v_t_new

```

تحلیل `keepdims=True`: در محاسبات برداری، $b^{[1]}$ یک بردار ستونی با ابعاد $(n_h, 1)$ است. اگر از `keepdims=True` استفاده نکنیم، دستور `np.sum` در پایتون یک آرایه یک‌بعدی با ابعاد $(n_h,)$ تولید می‌کند. اگرچه پایتون معمولاً می‌تواند این ابعاد را Broadcast کند، اما در شبکه‌های پیچیده‌تر یا در به‌روزرسانی‌های ماتریسی، این بی‌دقتی ابعادی می‌تواند منجر به جمع‌شدن اشتباه درک محورها (Axis) و باگ‌های خاموش (Silent Bugs) در طول عملیات انتشار رو به جلو ($Z = WX + b$) شود.

پاسخ سوال ۴: ترکیب Dropout و استنتاج

الف) در فاز آموزش: در هر تکرار آموزش، این نورون با احتمال $1 - p = 0.5$ مقدار $a = 10$ را عبور می‌دهد و با احتمال $p = 0.5$ خاموش شده (مقدار صفر) می‌شود. بنابراین، مقدار مورد انتظار (امید ریاضی) خروجی آن برابر است با:

$$\mathbb{E}[a] = (0.5 \times 10) + (0.5 \times 0) = 5$$

ب) در فاز تست (Inference): در زمان تست، ما هیچ نورونی را خاموش نمی‌کنیم تا از تمام ظرفیت یادگیری شبکه استفاده کنیم. اگر خروجی مستقیماً $a = 10$ ارسال شود، لایه بعدی مقداری دو برابر آنچه در زمان آموزش به آن عادت کرده بود (یعنی ۵) دریافت می‌کند. این تضاد، توزیع داده‌ها را به هم می‌ریزد. برای رفع این مشکل و سازگاری خروجی مورد انتظار، باید خروجی‌ها (یا اوزان مرتبط) را در زمان تست در احتمال نگه‌داشتن نورون (یعنی $1 - p = 0.5$) ضرب کنیم. بنابراین در فاز تست، مقدار $10 \times 0.5 = 5$ به لایه بعد ارسال می‌شود. (در پیاده‌سازی‌های مدرن مانند PyTorch، معمولاً این مقیاس‌دهی به صورت معکوس (Inverted Dropout) در همان زمان آموزش انجام می‌شود تا فاز تست نیازی به ضرب اضافه نداشته باشد).

پاسخ سوال ۵: برنامه‌نویسی

پاسخ فاز اول: اثبات بن‌بست مدل‌های خطی

کد پیاده‌سازی:

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import accuracy_score
3
4 # Train a linear model
5 linear_model = LogisticRegression()
6 linear_model.fit(X_train, y_train)
7 y_pred_linear = linear_model.predict(X_test)
8
9 # Evaluate
10 acc_linear = accuracy_score(y_test, y_pred_linear)
11 print(f"Linear Model Accuracy: {acc_linear * 100:.2f}%")

```

پاسخ چالشی و تحلیل هندسی: دقت این مدل روی داده‌های تست در حدود ۵۰٪ (معادل حدس تصادفی) خواهد بود. دلیل ریاضی و هندسی این شکست آن است که داده‌های حلقوی (تودرتو) دارای ویژگی تفکیک‌پذیری خطی (Linear Separability) نیستند. یک مدل خطی تنها می‌تواند یک ابرصفحه (در فضای دو بعدی، یک خط راست با معادله $w_1x_1 + w_2x_2 + b = 0$) رسم کند. هیچ خط راستی وجود ندارد که بتواند یک حلقه را به طور کامل از مرکز و محیط بیرون آن جدا کند، بنابراین مدل خطی کاملاً در این مسئله شکست می‌خورد.

پاسخ فاز دوم: چالش معماری کمینه (The Minimalist Architect)

کد پیاده‌سازی و تست تعداد نورون‌ها:

```
1 from sklearn.neural_network import MLPClassifier
2
3 # Testing different numbers of neurons in a single hidden layer
4 for n in [1, 2, 3, 4, 6]:
5     mlp_minimal = MLPClassifier(hidden_layer_sizes=(n,), activation='relu',
6                                 max_iter=3000, random_state=42)
7     mlp_minimal.fit(X_train, y_train)
8     acc = mlp_minimal.score(X_test, y_test)
9     print(f"Neurons: {n} -> Accuracy: {acc * 100:.2f}%")
```

پاسخ چالشی و تحلیل هندسی:

- کمترین تعداد نورون: به صورت تئوری و عملی، برای محصور کردن یک فضای بسته (یک چندضلعی) در دو بُعد با استفاده از ReLU، به حداقل ۳ یا ۴ نورون نیاز داریم. اما چون هدف ما استخراج یک حلقه (شکل غیرمحدب) است، معمولاً با ۴ الی ۶ نورون در یک لایه پنهان، شبکه می‌تواند با دقت بالای ۹۵٪ همگرایی پیدا کند.
- چرا ۱ یا ۲ نورون شکست می‌خورد؟ هر نورون با تابع ReLU معادل رسم یک خط (ابرفصله) است که فضا را به دو نیمه تقسیم می‌کند (یک سمت صفر و سمت دیگر خطی). با ۱ نورون تنها یک نیم‌صفحه داریم. با ۲ نورون، می‌توانیم یک نوار یا یک زاویه باز بسازیم. در هیچ‌کدام از این حالت‌ها نمی‌توان یک منطقه بسته و محصور در فضای دو بعدی ایجاد کرد تا سیارات کلاس ۱ در آن قرار گیرند. با اضافه شدن نورون ۳ (مثلاً) و ۴ (چهارضلعی)، فضای بسته شکل می‌گیرد و ترکیب آن‌ها توسط لایه خروجی می‌تواند یک "تپه" حلقوی ایجاد کند.

پاسخ فاز سوم: دام توابع فعال‌سازی (Activation Trap)

کد پیاده‌سازی و رسم منحنی افت خطا:

```
1 # Train deep MLP with Sigmoid
2 mlp_sigmoid = MLPClassifier(hidden_layer_sizes=(10, 10, 10), activation='logistic',
3                             learning_rate_init=0.01, max_iter=500,
4                             random_state=42)
5 mlp_sigmoid.fit(X_train, y_train)
6
7 # Train deep MLP with ReLU
8 mlp_relu = MLPClassifier(hidden_layer_sizes=(10, 10, 10), activation='relu',
9                          learning_rate_init=0.01, max_iter=500,
10                          random_state=42)
```

```

9 mlp_relu.fit(X_train, y_train)
10
11 # Plotting the loss curves
12 plt.plot(mlp_sigmoid.loss_curve_, label='Sigmoid Loss', color='red')
13 plt.plot(mlp_relu.loss_curve_, label='ReLU Loss', color='green')
14 plt.xlabel('Epochs')
15 plt.ylabel('Loss (Cross-Entropy)')
16 plt.title('Loss Curve: Sigmoid vs ReLU in Deep MLP')
17 plt.legend()
18 plt.show()

```

پاسخ چالشی و تحلیل ریاضی پدیده: با رسم این منحنی، مشاهده می‌کنیم که خطای شبکه با تابع ReLU در همان Epoch های اولیه به سرعت سقوط کرده و به صفر نزدیک می‌شود (همگرایی سریع). اما خطای شبکه با تابع Sigmoid تقریباً ثابت می‌ماند یا با سرعت بسیار کمتری کاهش می‌یابد.

این پدیده دقیقاً نشان‌دهنده مشکل محوشدگی گرادیان (Vanishing Gradient Problem) است. مشتق تابع Sigmoid حداکثر برابر با 0.25 است. وقتی خطای لایه خروجی از طریق الگوریتم پس‌انتشار (Backpropagation) به لایه‌های اولیه می‌رسد، طبق قاعده زنجیره‌ای، این گرادیان‌ها در هم ضرب می‌شوند. برای ۳ لایه پنهان، گرادیان حداقل در ضرب $(0.25)^3 = 0.015$ ضرب می‌شود. در نتیجه، گرادیان به لایه‌های اولیه نمی‌رسد، وزن‌های آن‌ها به‌روز نمی‌شوند و شبکه یادگیری را متوقف می‌کند. در مقابل، مشتق ReLU برای بخش مثبت برابر با ۱ است، لذا گرادیان بدون کاهش مقیاس (تضعیف) در طول لایه‌های عمیق جریان می‌یابد.